Lab 1: Digital Differential Analyzer (DDA)

Adding graphics package in Dev C++

1. Download the Dev C++ also download *https://tinyurl.com/3ve35mzh*
2. You need to copy "graphics.h" and "winbgim.h" into include the directory of Dev-Cpp program resource files. The exact directory address is –
   C:\Program Files (x86)\Dev-Cpp\MinGW64\include
3. Next, copy "libbgi.a" to the lib folder, which should be inside MinGW64. The directory address is –
   C:\Program Files (x86)\Dev-Cpp\MinGW64\lib
4. In Dev C++, Set it to the latest **TDM - GCC [version] 32-bit release**.
5. In Dev C++, Expand tools and select Compiler Options.
   - Ensure the "Add the following commands when calling the linker" check box is selected.
   - Then add these linkers in the input box – (copy and paste this line)
     -static-libgcc -lbgi -lgdi32 -lcomdlg32 -luuid -loleaut32 -lole32
   - Then click on OK to save.

LAB SHEET

Theory: Briefly explain DDA algorithm with its algorithm

Lab works:

Question 1: Write a code to draw a line using the DDA algorithm.

```
#include<graphics.h>

#include<math.h>

#include<conio.h>

int main()

{

        int x0,y0,x1,y1,i=0;

        float delx,dely,len,x,y;

        int gr=DETECT,gm;

        initgraph(&gr,&gm, NULL);

        printf("\n****** DDA Line Drawing Algorithm ***********");
```

```
printf("\n Please enter the starting coordinate of x, y = ");
scanf("%d %d",&x0,&y0);
printf("\n Enter the final coordinate of x, y = ");
scanf("%d %d",&x1,&y1);
dely=abs(y1-y0);
delx=abs(x1-x0);
if(delx<dely)
{
        len = dely;
}
else
{
        len=delx;
}
delx=(x1-x0)/len;
dely=(y1-y0)/len;
x=x0+0.5;
y=y0+0.5;
do{
        putpixel(x,y,3);
        x=x+delx;
        y=y+dely;
        i++;
        delay(30);
}while(i<=len);
```

```
        getch();
        closegraph();
        return 0;
}
```

Question 2: Write a code to draw a checker box using the DDA algorithm.

```
#include <stdio.h>
#include <graphics.h>
#include <math.h>

void drawCheckerboard(int rows, int cols, int cellSize) {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);
    int i, j;
    int x1, y1, x2, y2;
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            x1 = j * cellSize;
            y1 = i * cellSize;
            x2 = x1 + cellSize;
            y2 = y1;
            int dx = x2 - x1;
            int dy = y2 - y1;
            int steps = abs(dx) > abs(dy) ? abs(dx) : abs(dy);
            float xIncrement = (float)dx / (float)steps;
            float yIncrement = (float)dy / (float)steps;
```

```
    float x = x1, y = y1;
    putpixel(round(x), round(y), WHITE);
    for (int k = 0; k < steps; k++) {
        x += xIncrement;
        y += yIncrement;
        putpixel(round(x), round(y), WHITE);
    }
    x1 = j * cellSize;
    y1 = i * cellSize;
    x2 = x1;
    y2 = y1 + cellSize;
    dx = x2 - x1;
    dy = y2 - y1;
    steps = abs(dx) > abs(dy) ? abs(dx) : abs(dy);
    xIncrement = (float)dx / (float)steps;
    yIncrement = (float)dy / (float)steps;
    x = x1;
    y = y1;
    putpixel(round(x), round(y), WHITE);
    for (int k = 0; k < steps; k++) {
        x += xIncrement;
        y += yIncrement;
        putpixel(round(x), round(y), WHITE);
    }
}
```

```
    }
    delay(5000);
    closegraph();
}
int main() {
    int rows = 8;
    int cols = 8;
    int cellSize = 64;
    drawCheckerboard(rows, cols, cellSize);
    return 0;
}
```

Conclusion

- What did u learn?